

GIORNO 10

Script realistici: il catalogo dei pattern dei giorni 01-09

Cosa saprete fare stasera

Scrivere uno script completo: carica, valida, elabora, riporta,
con main() che dirige e un controllo finale che i conti tornino

Adattare uno script a un caso nuovo cambiando solo le righe marcate

Leggere un problema d'ufficio in tre righe e dire quali pattern servono, e perché

Validare dati da tastiera e da file: prevenire con un if, gestire con un try stretto

Calcolare totali, medie con il caso zero, massimi con il loro proprietario,
conteggi e somme per categoria, classifiche

Produrre lo stesso report a video e su file, con registro e scarti
che ripartono puliti a ogni lancio

Trovare, in uno script che gira senza errori, il risultato sbagliato,
e ricondurlo all'errore tipico del suo pattern

Il lavoro di oggi non è imparare pezzi: è sceglierli e montarli.

1. RIPASSO E DOMANDE: IL CATALOGO E COME SI USA

Sei voci per scheda, e un programma con un pezzo di ogni giornata

1.2 Pattern → giornate: da dove viene ogni pezzo (1/2)

Giornata	Pattern che la giornata introduce o usa in modo esplicito	Quanti
01	P1 P2 P33	3
02	P1 P2 P3 P4 P5 P6 P18	7
03	P2 P3 P4 P5 P6 P7 P9 P18 P34	9
04	P4 P5 P7 P10 P11 P14 P24	7
05	P4 P9 P13 P14 P15 P16 P19 P20 P32	9

La tabella della teoria letta per colonna: per ogni giornata, i pattern di cui avete già scritto i pezzi. Le prime due sono le più leggere: danno la forma dello script e i tipi dei dati, non gli algoritmi.

1.2 Pattern → giornate: da dove viene ogni pezzo (2/2)

Giornata	Pattern che la giornata introduce o usa in modo esplicito	Quanti
06	P1 P8 P9 P10 P14 P16 P19 P23 P24 P27 P28 P34	12
07	P6 P11 P12 P15 P16 P17 P18 P19 P20 P21 P22 P26 P27 P32	14
08	P4 P6 P11 P13 P17 P20 P21 P22 P24 P25 P26 P27 P28 P29 P30 P31 P33 P34	18
09	P5 P6 P8 P9 P10 P12 P14 P15 P25 P26 P27 P29 P30 P32 P33 P34	16

Nessuna giornata resta scoperta. I numeri di paragrafo precisi stanno nella voce «Da dove viene» di ogni scheda: su ogni pattern la domanda è «dove l'avete già visto?».

1.3 Perimetro della giornata: nessun costruito nuovo

```
import json

from pathlib import Path


TARIFFA_KM = 0.42 # 01 costante in cima
SOGLIA_LUNGA = 100 # 01 nessun numero magico
TRASFERTE = [("Salerno", "128"), ("Napoli", " 54 "), ("Avellino", "sei")] # 07

def rimborso(km): # 06 funzione che restituisce
    return round(km * TARIFFA_KM, 2)

totale = 0.0 # 05 accumulatore
for citta, testo in TRASFERTE: # 05 ciclo, 03 unpacking
    try:
        km = int(testo) # 03 conversione, 09 try stretto
    except ValueError:
        print(f"[!] {citta}: km non numerici '{testo}'")
        continue

    if km > SOGLIA_LUNGA: # 04 decisione
        tipo = "lunga"
    else:
        tipo = "breve"

    totale += rimborso(km)

    print(f"[OK] {citta:<8}{km:>4} km {tipo:<6}{rimborso(km):>6.2f}") # 02

percorso = Path("rimborsi.json") # 08 file e JSON
percorso.write_text(json.dumps({"totale": round(totale, 2)}), encoding="utf-8")

print("Salvato:", percorso.read_text(encoding="utf-8"))
```

1.3 L'output del programma con un pezzo per giornata

```
[OK] Salerno 128 km lunga 53.76  
[OK] Napoli 54 km breve 22.68  
[!] Avellino: km non numerici 'sei'  
Salvato: {"totale": 76.44}
```

TRABOCCHETTO: " 54 " passa senza strip(), perché int() tollera gli spazi ai bordi (Giorno 09 §8.3). Un .isdecimal() lo avrebbe rifiutato. Il lavoro di oggi non è imparare pezzi: è sceglierli e montarli.

1.4 Come si adatta uno schema: le righe marcate # <-- adattare

```
# ===== PARTE GENERALE: si riusa cosi' com'e' =====

def conta_oltre(valori, soglia):

    """Restituisce quanti valori superano la soglia, bordo escluso."""

    quanti = 0

    for valore in valori:

        if valore > soglia:

            quanti += 1

    return quanti


# ===== PARTE SPECIFICA: si cambia per un altro caso =====

VALORI = [72, 88, 65, 91, 70, 84, 59, 67]      # <-- adattare

SOGLIA = 80                                  # <-- adattare

ETICHETTA = "Punteggi del turno"             # <-- adattare


# TRABOCCHETTO: la soglia compare nell'output tramite la costante, non
#   riscritta a mano nell'etichetta. "Punteggi oltre 80" scritto nel testo
#   resterebbe 80 anche dopo aver portato SOGLIA a 30.

print(f"{ETICHETTA} oltre {SOGLIA}: {conta_oltre(VALORI, SOGLIA)} su {len(VALORI)}")
```

1.4 Lo stesso schema su due casi: tre righe toccate

Riga marcata	Caso 1: punteggi Aurora	Caso 2: colli LogiSud
VALORI	[72, 88, 65, 91, 70, 84, 59, 67]	[12.5, 3.0, 31.0, 7.2, 45.0]
SOGLIA	80	30
ETICHETTA	"Punteggi del turno"	"Colli in kg"
Output	Punteggi del turno oltre 80: 3 su 8	Colli in kg oltre 30: 2 su 5

Tre righe toccate, la funzione identica. È la forma della variante di P34, e il cuore dell'ultimo esercizio per casa. Le soluzioni di oggi separano le due zone con un commento a riquadro, e ogni traccia ha una sezione PER RIUSARLO.

2. L'USCITA: LA PAGINA CHE SI LEGGE

P1-P3 · Un programma giusto con un output illeggibile torna indietro

2.1 P1 — Lo scheletro dello script

```
"""Che cosa fa lo script, da dove prende i dati, che cosa produce."""

VALORE_DI_PROVA = 3                                # <-- adattare
FATTORE = 1.22                                     # <-- adattare

def calcola(valore):                                # <-- adattare
    """Restituisce il risultato del calcolo: non stampa niente."""
    return valore * FATTORE

def main():
    """Il regista: chiama le funzioni in ordine, i conti li fanno loro."""
    risultato = calcola(VALORE_DI_PROVA)
    print(f"Risultato: {risultato:.2f}")             # <-- adattare

if __name__ == "__main__":
    main()
```

2.2 P2 — La riga di report: etichetta, valore, larghezza, decimali

```
COLONNA_TESTO = 20                                # <-- adattare
COLONNA_NUMERO = 10                                # <-- adattare

etichetta = "Pratiche evase"                       # <-- adattare
valore = 97                                         # <-- adattare
parte, intero = 97, 128                            # <-- adattare

# Testo a sinistra, numero a destra: le righe una sotto l'altra si incolonnano.
print(f"{etichetta:<{COLONNA_TESTO}}{valore:>{COLONNA_NUMERO}}")

# La quota: .1% vuole la FRAZIONE, moltiplica per 100 da solo.
print(f"{'Quota':<{COLONNA_TESTO}}{parte / intero:>{COLONNA_NUMERO}.1%}")
```

Quando serve: ogni numero che esce a video o su file, con l'etichetta, a larghezza fissa, con i decimali giusti. Variante: il numero è già una percentuale? `f"{75.8:.1f}%"`, con il simbolo scritto a mano.

2.2 P2 in azione — Il riepilogo dello sportello di Villanova

Pratiche ricevute	128
Pratiche evase	97
Quota evasa	75.8%
Diritti incassati	1,234.50
Quota sbagliata	7578.1%

TRABOCCHETTO: .1% su un numero già moltiplicato per 100 lo moltiplica di nuovo, e nessuno protesta. Il :, mette la virgola delle migliaia all'inglese: in un report per l'ufficio va convertito con P5. Chicca: f"{etichetta:.<24}" riempie di puntini.

2.3 P3 — La tabella a larghezza fissa e il testo che sfonda

```
COL_CHIAVE = 8                                # <-- adattare
COL_TESTO = 12                                # <-- adattare
COL_NUMERO = 10                               # <-- adattare

LARGHEZZA_TABELLA = COL_CHIAVE + COL_TESTO + COL_NUMERO

righe = [("A", "testo", 1.0)]                 # <-- adattare

print(f"{'CHIAVE':<{COL_CHIAVE}}{'TESTO':<{COL_TESTO}}{'NUMERO':>{COL_NUMERO}}")
print("-" * LARGHEZZA_TABELLA)

for chiave, testo, numero in righe:
    # Lo slicing taglia il testo lungo; sotto la larghezza non toglie niente.
    print(f"{chiave:<{COL_CHIAVE}}{testo[:COL_TESTO]:<{COL_TESTO}}{
        f"{numero:>{COL_NUMERO}.2f}"})
```

2.3 P3 in azione — Il listino LogiSud, e la zona che non sta in colonna

ZONA	DESCRIZIONE	PREZZO
NORD	Nord Italia	8.50
CENTRO	Centro	7.00
SUD	Sud Italia	9.50
ISOLE	Isole maggio	14.00

Senza fetta la riga ISOLE sarebbe di 41 caratteri, non 30.

TRABOCCHETTO: :<12 NON taglia. Una larghezza è un minimo, non un massimo: senza la fetta la riga ISOLE si allunga e il prezzo esce di colonna. E i numeri allineati a sinistra fanno sembrare 14.00 più piccolo di 8.50.

3. L'INGRESSO: DAL TESTO GREZZO AL DATO

P4-P6 · Il dato non arriva mai pulito

3.1 P4 — Normalizzare un testo: strip, maiuscole, simboli interni

```
def normalizza_codice(testo):  
    """Restituisce il codice senza spazi ai bordi, in maiuscolo."""  
    return testo.strip().upper()  
  
def normalizza_nome(testo):  
    """Restituisce il nome con un solo spazio fra le parole, iniziali maiuscole."""  
    return " ".join(testo.split()).title()  
  
# I simboli interni (spazi, trattini, punti) si tolgono uno per uno.  
codice = normalizza_codice(testo_digitato).replace(" ", "")    # <-- adattare
```

Quando serve: prima di ogni confronto, ricerca o conteggio su un testo scritto da una persona. "REFERTO", "referto " e " Referto" sono tre stringhe per Python e una sola categoria per l'ufficio. Le due funzioni non si scambiano.

3.1 P4 in azione — Il modulo TalentHub scritto di fretta

```
[Rossi Anna]  
[IT60X0542811101000000123456] 27 caratteri  
D'ANGELO -> D'Angelo  
MCDONALD -> Mcdonald  
False True
```

*Le quadre sono un trucco da collaudo: mostrano gli spazi ai bordi, se ne sono rimasti. .title() rende bene D'ANGELO e non MCDONALD: è un limite, si dichiara.
L'ultima riga è il TRABOCCHETTO: normalizzato un lato solo, il confronto dice False.*

3.2 P5 — Il numero scritto all'italiana: virgola e conversione

```
testo = "1.250,00"                                # <-- adattare
pulito = testo.strip()
if "," in pulito:
    # L'ordine conta: prima si tolgono i punti, poi la virgola diventa punto.
    pulito = pulito.replace(".", "").replace(",", ".")
importo = float(pulito)
```

Quando serve: ogni importo scritto in Italia, 12,50 o 1.250,00. Per float() la virgola non è un numero, e il punto delle migliaia è un decimale. La regola in una riga: se c'è la virgola, prima via i punti, poi la virgola diventa punto.

3.2 P5 in azione — Tre importi, con e senza la regola

```
12,50 -> 12.50 senza la regola: 12.50
1.250,00 -> 1250.00 senza la regola: 1250.00
12.50 -> 12.50 senza la regola: 1250.00
```

TRABOCCHETTO: togliere i punti sempre. "12.50" scritto all'inglese perde il punto e diventa 1250.0, cento volte tanto, senza errori. In uscita il contrario: prima la f-string con il punto, poi il punto diventa virgola, e con le migliaia servono tre replace.

3.3 P6 — La riga in campi: split, conteggio dei campi, unpacking

```
SEPARATORE = ";"

CAMPI_ATTESI = 3                                # <-- adattare

riga = "valore uno ; valore due ; 3.5"          # <-- adattare
campi = riga.split(SEPARATORE)

# 1. Prima si contano i campi: l'unpacking viene DOPO il controllo.
if len(campi) != CAMPI_ATTESI:
    print(f"[!] servono {CAMPI_ATTESI} campi, ne ha {len(campi)}")
else:
    primo, secondo, terzo = campi                # <-- adattare: i nomi

    # 2. Poi si pulisce ogni campo, uno per uno.
    primo = primo.strip()
    secondo = secondo.strip().upper()
    terzo = float(terzo)
```

4. VALIDARE

P7-P9 · Il dato pulito può essere ancora sbagliato: accettarlo, o dire perché no

4.1 P7 — I quattro controlli e la variabile errore

```
MINIMO = 18                                # <-- adattare

MASSIMO = 67                              # <-- adattare

AMMESSI = ["SALERNO", "NAPOLI", "REMOTO"]  # <-- adattare


def motivo_rifiuto(testo_numero, testo_scelta):

    """Restituisce "" se i dati vanno bene, altrimenti il motivo del rifiuto."""

    numero = testo_numero.strip()

    scelta = testo_scelta.strip().upper()

    errore = ""

    if not numero:                          # 1. presenza
        errore = "campo vuoto"             # <-- adattare i messaggi

    elif not numero.isdecimal():            # 2. forma
        errore = f"'{numero}' non è un numero"

    elif not MINIMO <= int(numero) <= MASSIMO: # 3. intervallo
        errore = f"{numero} fuori intervallo {MINIMO}-{MASSIMO}"

    elif scelta not in AMMESSI:             # 4. valori ammessi
        errore = f"'{scelta}' non ammesso"

    return errore
```

4.1 P7 in azione — Cinque prove sul modulo di candidatura

[OK] candidato di 34 anni, sede REMOTO

[ERRORE] età: 'trenta' non è un numero. Scrivete gli anni in cifre

[ERRORE] età: 15 fuori intervallo. Ammessi da 18 a 67

[ERRORE] età: campo vuoto. Scrivete gli anni in cifre

[ERRORE] sede: 'MILANO' non prevista. Scegliete fra SALERNO, NAPOLI, REMOTO

Ogni messaggio dice le tre cose del Giorno 04: che cosa non va, in quale campo, che cosa fare. Il messaggio generico «dato non valido» lascia il candidato senza sapere quale campo correggere, né come.

4.2 P8 — La funzione che converte o solleva ValueError

```
def converti_intero(testo, minimo, massimo):  
    """Restituisce l'intero; solleva ValueError per forma o intervallo."""  
    pulito = testo.strip()  
    if not pulito.isdecimal():  
        raise ValueError(f"'{testo}' non è un numero intero")      # <-- adattare  
    valore = int(pulito)  
    if not minimo <= valore <= massimo:  
        raise ValueError(f"{valore} fuori intervallo: da {minimo} a {massimo}")  
    return valore  
  
# Il chiamante: il try e' stretto sulla sola chiamata che puo' sollevare.  
try:  
    colli = converti_intero("7", 1, 50)                             # <-- adattare  
except ValueError as e:  
    print(f"[ERRORE] {e}")
```

4.3 P9 — Chiedere finché il dato è buono, con un tetto

```
TENTATIVI_MASSIMI = 3

NESSUN_VALORE = -1


def chiedi_intero(prompt, minimo, massimo):

    """Chiede un intero fra minimo e massimo; oltre il tetto, NESSUN_VALORE."""

    for tentativo in range(TENTATIVI_MASSIMI):

        testo = input(prompt)

        try:

            valore = int(testo)

        except ValueError:

            print("[ERRORE] Serve un numero intero.")          # <-- adattare

            continue

        if minimo <= valore <= massimo:

            return valore

    print(f"[ERRORE] Ammessi da {minimo} a {massimo}.")          # <-- adattare

print("[!] Troppi tentativi.")                                  # <-- adattare

return NESSUN_VALORE
```

4.3 P9 in azione — I colli allo sportello LogiSud, risposte simulate

```
Colli da spedire: zero
[ERRORE] Serve un numero intero, per esempio 7.
Colli da spedire: 0
[ERRORE] Ammessi da 1 a 50 colli.
Colli da spedire: 7
Registrati 7 colli.
Colli da spedire: tanti
[ERRORE] Serve un numero intero, per esempio 7.
Colli da spedire: 99
[ERRORE] Ammessi da 1 a 50 colli.
Colli da spedire: -1
[ERRORE] Ammessi da 1 a 50 colli.
[!] Troppi tentativi: la spedizione passa all'operatore.
Nessun valore registrato.
```

Il "5" del secondo giro era buono, ma arriva dopo il tetto. TRABOCCHETTO: senza il continue nel gestore, al primo "zero" il confronto trova valore mai assegnato: `UnboundLocalError` dentro la funzione.

5. DECIDERE

P10-P12 · Le regole diventano dati: cambia la delibera, la logica resta

5.1 P10 — La decisione a soglie: una lista di tuple

```
SOGLIE = [(8000, "fascia 1"), (16000, "fascia 2")] # <-- adattare: crescenti
OLTRE = "fascia 3"                                # <-- adattare
```

```
def fascia(valore, soglie, oltre):
    """Restituisce l'esito della prima fascia con valore <= limite, o oltre."""
    # Le coppie (limite, esito) vanno in ordine CRESCENTE di limite.
    for limite, esito in soglie:
        if valore <= limite:
            return esito
    return oltre
```

Quando serve: un valore numerico cade in una fascia, ISEE e quota, consumo e tariffa. Le fasce hanno dei bordi, ed è lì che si sbaglia. Con due o tre soglie fisse la catena di elif del Giorno 04 si legge meglio, e va benissimo.

5.1 P10 in azione — La mensa scolastica, collaudata sui bordi

```
ISEE    7999    quota mensile    15.00
ISEE    8000    quota mensile    15.00
ISEE    8001    quota mensile    30.00
ISEE    28000   quota mensile    45.00
ISEE    28001   quota mensile    60.00
Soglie in disordine, ISEE 7999: quota 45.00
```

Il collaudo si fa sui bordi: un euro sotto, sul limite, un euro sopra. TRABOCCHETTO nell'ultima riga: soglie in disordine, la prima coppia è (28000, 45.00) e si prende tutto. E con < al posto di <= gli 8.000 esatti pagano 30.00.

5.2 P11 — Il punteggio a indizi: pesi, somma, soglia

```
PESI = {"parola spia": 3, "altra parola": 2}      # <-- adattare
SOGLIA = 5                                       # <-- adattare

def punteggio(testo, pesi):
    """Restituisce la somma dei pesi delle parole spia trovate nel testo."""
    testo = testo.lower()
    totale = 0
    for parola, peso in pesi.items():
        if parola in testo:
            totale += peso
    return totale

# Dalla soglia in su il testo e' segnalato: >= e non >, come nel Giorno 04.
segnalato = punteggio(testo, PESI) >= SOGLIA    # <-- adattare: il testo
```

5.2 P11 in azione — Il tariffario delle parole spia

0 punti	ok	Le confermo l'appuntamento in sede
3 punti	ok	Urgente: pratica di rimborso in scadenza
9 punti	SPAM	Offerta imperdibile: clicca subito e ricevi un buono gratis
3 punti	ok	Nuovo bando della provincia per i tirocini

TRABOCCHETTO: l'ultimo messaggio prende 3 punti senza nessuna parola spia. "vinci" sta dentro "provincia": in cerca una sequenza di caratteri, non una parola. Resta sotto soglia per fortuna, non per merito. Chicca: `.count()` pesa le ripetizioni.

5.3 P12 — La tabella di decisione in un dizionario, con .get()

```
TABELLA = {"VALORE_A": "esito A", "VALORE_B": "esito B"}    # <-- adattare  
RIPIEGO = "esito di tutti gli altri casi"                  # <-- adattare
```

```
def decidi(chiave, tabella, ripiego):  
    """Restituisce il valore associato alla chiave normalizzata, o il ripiego."""  
    return tabella.get(chiave.strip().upper(), ripiego)
```

Quando serve: a ogni valore di un campo corrisponde un valore fisso, categoria e ufficio, zona e tariffa. Dieci elif sulla stessa variabile sono una tabella travestita da codice. Se la chiave assente è un'anomalia, si scarta con il motivo.

5.3 P12 in azione — Lo smistamento dei ticket Aurora

```
REFERTO      -> Archivio referti
PAGAMENTO    -> Cassa
ALTRO        -> Centralino
VACCINI      -> Centralino
RECLAMO      -> URP
Senza normalizzare: Centralino
```

TRABOCCHETTO: senza normalizzare, "pagamento" va al centralino in silenzio, ed è il più insidioso perché non si vede. Con `UFFICI["VACCINI"]` al posto di `.get()`, invece, il programma si ferma con `KeyError`.

6. RIPETERE

P13-P14 · Un ciclo, e un segnale d'uscita scelto bene

6.1 P13 — La sentinella: leggere finché arriva il segnale di fine

```
SENTINELLA = "FINE"                                # <-- adattare

PROMPT = "Codice (FINE per chiudere): "            # <-- adattare

codici = []

# 1. Prima lettura FUORI dal ciclo; 2. rilettura in CODA al corpo.
testo = input(PROMPT).strip().upper()

while testo != SENTINELLA:

    codici.append(testo)                             # <-- adattare: il lavoro

    testo = input(PROMPT).strip().upper()

print(f"Letti {len(codici)} codici")
```

Quando serve: dati di quantità ignota. Il magazziniere batte i codici dei colli finché ne ha, e nessuno sa dire prima quanti saranno. Da un file, `readline()` restituisce la stringa vuota a fine file: la sentinella che il disco vi dà gratis.

6.1 P13 in azione — La sentinella arriva scritta male

```
Codice collo: ls01
Codice collo: LS02
Codice collo:  Fine
Codici registrati: 2 -> ['LS01', 'LS02']
Risposte mai lette: 1
```

La sentinella arriva come " Fine" e il ciclo si ferma lo stesso, perché si confronta dopo la normalizzazione. TRABOCCHETTO: confrontando la risposta grezza, il ciclo tira dritto, legge LS03 e finisce con IndexError.

6.2 P14 — Il menu a scelta: while True, una funzione per voce

```
def voce_uno():                                # <-- adattare: una per voce
    """Il lavoro della voce 1 sta qui, non dentro il ciclo."""
    print("Voce uno eseguita.")

def main():
    while True:
        print("1) Prima voce    0) Esci")          # <-- adattare
        scelta = input("Scelta: ").strip()
        if scelta == "1":
            voce_uno()
        elif scelta == "0":
            print("Arrivederci.")
            break
        else:
            print(f"[!] Scelta non valida: {scelta}")
```

6.2 P14 in azione — Lo sportello anagrafe di Villanova

1) Certificato di residenza

2) Stato di una pratica

3) Orari dello sportello

0) Esci

Scelta (1-3, 0 per uscire): 1

Certificato di residenza: pronto in giornata.

Scelta (1-3, 0 per uscire): 3

Sportello aperto la mattina, dalle 8.30 alle 12.30.

Scelta (1-3, 0 per uscire): 9

[!] Scelta non valida: 9

Scelta (1-3, 0 per uscire): 0

Sportello chiuso.

Il menu lo stampa il dizionario delle voci: una voce nuova è una riga nuova. TRABOCCHETTO: il confronto è con "1" fra virgolette. Con 1 intero nessuna voce scatta mai, senza errori: input() restituisce sempre una stringa.

7. ACCUMULARE E CONTARE

P15-P18 · Inizio fuori dal ciclo, aggiornamento dentro, lettura dopo

7.1 P15 — Totale, conteggio e media, con il caso zero elementi

```
def media(valori):  
    """Restituisce la media. Precondizione: la lista non e' vuota."""  
    return sum(valori) / len(valori)  
  
# Il caso zero lo controlla CHI CHIAMA, prima della divisione.  
  
if valori:                                     # <-- adattare: la lista  
    print(f"Media: {media(valori):.2f}")  
else:  
    print("Nessun dato: media non calcolabile")    # <-- adattare
```

Quando serve: ogni riepilogo numerico, che prima o poi gira su un turno vuoto. In azione: somma 596, media 74.50; il turno vuoto dice «non calcolabile»; sopra 70 sono 4 valori, media 83.75. Con >= entra anche il 70 sul bordo: 5 valori, 81.00.

7.2 P16 — Massimo e minimo, e chi li possiede

```
coppie = [(3.5, "A"), (7.0, "B")]          # <-- adattare: (valore, chi)

# Si parte dal PRIMO elemento, non da 0: vale anche con numeri negativi.

minimo = coppie[0]

massimo = coppie[0]

for coppia in coppie[1:]:

    # Si confronta il primo campo, si conserva la coppia intera.

    if coppia[0] < minimo[0]:

        minimo = coppia

    if coppia[0] > massimo[0]:

        massimo = coppia
```

Quando serve: «qual è l'ordine più alto?» chiede anche «di chi è». In azione: più alto NS-1010, 278.90; più basso NS-1008, 27.00. Con minimo = 0 il minimo non scende mai. Chicca: max() su coppie restituisce la coppia intera.

7.3 P17 — Il contatore per categoria: `.get(k, 0) + 1`

```
def conta_per_chiave(chiavi):  
    """Restituisce il dizionario chiave -> quante volte compare."""  
    conteggi = {}  
    for chiave in chiavi:  
        # .get(chiave, 0): la chiave nuova parte da zero, senza KeyError.  
        conteggi[chiave] = conteggi.get(chiave, 0) + 1  
    return conteggi
```

Quando serve: « quanti per ciascun tipo », con tipi ignoti in anticipo. Con `conteggi[chiave] += 1` su una chiave nuova arriva `KeyError` alla prima comparsa. La funzione non sa niente di maiuscole: si normalizza prima di chiamarla.

7.3 P17 in azione — Sette ticket con le maiuscole a caso

```
PRENOTAZIONE    0
REFERTO         3
PAGAMENTO       2
RECLAMO         1
ALTRO           1
6 categorie contando i dati grezzi
```

Con le categorie dichiarate compaiono anche quelle a zero, nell'ordine voluto. TRABOCCHETTO nell'ultima riga: contando i dati grezzi "referto " e "Pagamento" diventano categorie a sé, e i conti si spezzano senza nessun errore.

7.4 P18 — Aggregare per chiave: somma, quota percentuale e barra

```
LARGHEZZA_BARRA = 40

somme = somma_per_chiave(coppie)                                # <-- adattare: le coppie

# Il totale si calcola DOPO: la quota ha bisogno di tutte le somme.

totale = sum(somme.values())

for chiave, importo in somme.items():

    quota = importo / totale

    barra = "#" * round(quota * LARGHEZZA_BARRA)

    print(f"{chiave:<12}{importo:>8.2f}{quota:>7.1%}   {barra}")      # <-- adattare
```

Quando serve: «quanto per ciascun tipo», con la quota e una barra. `somma_per_chiave()` è il contatore di P17 con + importo al posto di + 1. La quota calcolata dentro il ciclo di accumulo usa un totale parziale.

7.4 P18 in azione — I dodici ordini buoni, per categoria

AUDIO	299.40	20.5%	#####
INFORMATICA	383.00	26.3%	#####
ACCESSORI	157.50	10.8%	####
VIDEO	617.70	42.4%	#####
TOTALE	1457.60		

La barra usa round() e non int(): arrotonda come la percentuale accanto. TRABOCCHETTO: int() tronca, INFORMATICA scenderebbe a 10 cancelletti, VIDEO a 16, e sotto il 2,5% una categoria non ne avrebbe nemmeno uno.

8. CERCARE, FILTRARE, ORDINARE

P19-P22 · Quattro domande del responsabile vendite, quattro pattern

8.1 P19 — Cercare il primo che soddisfa, e il «non trovato»

```
def cerca_per_codice(elenco, codice):  
    """Restituisce il primo record con quel codice, oppure None."""  
    # Il codice cercato si normalizza come quelli dell'elenco (P4).  
    codice = codice.strip().upper()  
    for record in elenco:  
        if record[0] == codice: # <-- adattare: il campo che fa da codice  
            # return esce dalla funzione, e con lei dal ciclo.  
            return record  
    # Qui si arriva solo se nessun giro ha trovato: DOPO il for, non dentro.  
    return None
```

Quando serve: trovare un record per codice, o il primo che rispetta una condizione. In azione: "ns-a03" trova la Webcam HD a 39.00, NS-Z99 «non a listino». Con else: return None dentro il for, la funzione si arrende al primo giro.

8.2 P20 — Filtrare e partizionare: i buoni e gli scarti con il motivo

```
buoni = []
scarti = []

for elemento in elementi: # <-- adattare

    # Il motivo riparte vuoto a ogni giro; if/elif: vince il primo controllo.
    motivo = ""

    if not controllo_1(elemento): # <-- adattare: i controlli del caso
        motivo = "primo motivo"

    elif not controllo_2(elemento): # <-- adattare

        motivo = "secondo motivo"

    if motivo:

        scarti.append((elemento, motivo))

    else:

        buoni.append(elemento)

assert len(buoni) + len(scarti) == len(elementi), "elementi persi nel conto"
```

8.2 P20 in azione — Quattro richieste di preventivo LogiSud

```
Buone: [('NORD', 12.5), ('CENTRO', 7.2)]  
[!] ESTERO;3      zona sconosciuta: ESTERO  
[!] SUD;venti    peso non numerico: 'venti'  
Richieste 4 = buone 2 + scartate 2
```

TRABOCCHETTO: con motivo = "" scritto una volta sola, prima del for, il motivo di SUD;venti resta attaccato al giro dopo: CENTRO;7.2 finisce fra gli scarti con il peso di un'altra riga. Buone 1, scarti 3.

8.3 P21 — Doppioni e confronto fra due elenchi

```
def senza_doppioni(elenco):  
    """Restituisce una lista nuova: ogni voce una volta, nell'ordine d'arrivo."""  
    visti = []  
    for voce in elenco:  
        voce = voce.strip().lower() # <-- adattare: la normalizzazione del caso  
        # in su una lista: vero se la voce c'e' gia' (Giorno 07 §5.1).  
        if voce not in visti:  
            visti.append(voce)  
    return visti
```

Quando serve: iscrizioni ripetute, due elenchi da unire, «chi c'è in tutti e due». In azione, la newsletter NovaStore: 4 iscritti senza doppioni, bianchi in comune. Senza .strip().lower() diventano 5, e Bianchi riceve la newsletter due volte.

8.4 P22 — La classifica: coppie (valore, nome) e i primi N

```
def primi(dizionario, quanti):  
    """Restituisce le prime coppie (valore, chiave), dalla piu' alta."""  
    coppie = []  
    for chiave, valore in dizionario.items():  
        # Il valore PRIMA: la tupla si ordina dal primo campo (Giorno 07 §8.4).  
        coppie.append((valore, chiave))  
    # sort() ordina sul posto e restituisce None: non si riassegna mai.  
    coppie.sort(reverse=True)  
    return coppie[:quanti]
```

Quando serve: «i tre candidati migliori», «le cinque parole più frequenti». La ricetta non cambia: valore davanti, ordinamento decrescente, fetta dei primi N. Con classifica = coppie.sort() la fetta trova None.

8.4 P22 in azione — I primi tre candidati TalentHub

1. Verdi Luca	91
2. Ricci Paolo	88
3. Gallo Pietro	83

TRABOCCHETTO: con le coppie (nome, valore) l'ordine lo decide il nome, e in testa escono Verdi, Rossi e Ricci con 91, 78, 88 che non scendono. Il caso da guardare apposta è la parità: reverse=True rovescia anche i nomi, ed è E6.

9. FUNZIONI CHE SI RIUSANO

P23-P24 · Se il calcolo sta in una `print()`, ogni richiesta è una riscrittura

9.1 P23 — Calcolare e presentare: `calcola_restituisce`, `stampa_stampa`

```
def calcola_costo(peso, zona, listino): # <-- adattare: nome e parametri
    """Restituisce il numero. Non stampa niente."""
    return listino[zona] * peso # <-- adattare: la formula del caso

def stampa_preventivo(codice, costo): # <-- adattare
    """Stampa il numero. Non calcola, non restituisce niente."""
    print(f"{codice} {costo:>10.2f} euro") # <-- adattare: il formato (P2)

# Chi chiama tiene il valore restituito, e lo usa quante volte vuole.
costo = calcola_costo(peso, zona, LISTINO)
stampa_preventivo(codice, costo)
```

9.1 P23 in azione — Il preventivo LogiSud e il tetto di spesa

```
LS-0510  SUD      7.2 kg    68.40 euro
LS-0511  ISOLE    3.0 kg    42.00 euro
Totale preventivi: 110.40 euro
[!] tetto di 100.00 superato di 10.40
```

Il valore restituito si usa tre volte: stampa, somma, confronto. TRABOCCHETTO: con print al posto del return la funzione stampa 68.4 nudo, restituisce None, e il programma si ferma una riga dopo con TypeError su NoneType.__format__.

9.2 P24 — Il validatore di record: (record, "") o (None, motivo)

```
def valida_record(campi): # <-- adattare: valida_<cosa>

    """Restituisce (record, "") se la riga regge, (None, motivo) se no."""

    # 1. Prima quanti campi: nessun indice prima di saperlo.

    if len(campi) != CAMPI_ATTESI:

        return None, f"servono {CAMPI_ATTESI} campi, ne ha {len(campi)}"

    codice, valore_testo = campi # <-- adattare: i nomi dei campi

    # 2. Poi forma e intervallo (P7), con il try stretto a una riga.

    try:

        valore = float(valore_testo) # <-- adattare: la conversione

    except ValueError:

        return None, f"valore non numerico: '{valore_testo}'"

    if not MINIMO <= valore <= MASSIMO: # <-- adattare

        return None, f"valore fuori intervallo: {valore}"

    return (codice.strip(), valore), ""
```

9.2 P24 in azione — Quattro righe di spedizione LogiSud

```
riga 1: [OK] ('LS-0601', 'NORD', 12.5)
riga 2: [!] servono 3 campi, ne ha 2
riga 3: [!] peso fuori intervallo: -4.0
riga 4: [OK] ('LS-0604', 'CENTRO', 7.2)
```

TRABOCCHETTO: scritto motivo, record = valida_spedizione(campi), nessun errore e tutto al contrario: «riga 1: [!]» con il record come motivo, «riga 2: [OK]» con il motivo come record. I due nomi seguono l'ordine del return.

10. LEGGERE DAI FILE

P25-P27 · Il file c'è e non si trova, le righe non tornano, il listino cambia

10.1 P25 — Le cartelle di lavoro accanto allo script, con Path(__file__)

```
from pathlib import Path

# La cartella dello script: non dipende da dove lo lanciate (Giorno 08 §7.4).
CARTELLA = Path(__file__).resolve().parent

DATI = CARTELLA / "dati" # <-- adattare: nomi delle cartelle

OUTPUT = CARTELLA / "output"

PERCORSO_INGRESSO = DATI / "ordini.txt" # <-- adattare: i file del caso

# output/ si crea da sola; parents=True crea anche i livelli intermedi.
OUTPUT.mkdir(parents=True, exist_ok=True)

if not PERCORSO_INGRESSO.exists():

    # A chi legge il nome del file, mai il percorso intero (Giorno 09 E9).
    print(f"[ERRORE] manca {PERCORSO_INGRESSO.name} nella cartella dati")
```

10.2 P26 — Il caricatore e il numero di riga dell'editor

```
SEPARATORE = ";" # <-- adattare
```

```
def carica_righe(percorso):  
    """Restituisce [(numero_riga, campi), ...] delle righe utili del file."""  
    righe = []  
    with open(percorso, encoding="utf-8") as f:  
        # enumerate(f, 1): la prima riga del file e' la 1, come nell'editor.  
        for numero, riga in enumerate(f, 1):  
            riga = riga.rstrip("\n")  
            if riga.strip() == "" or riga.startswith("#"): # <-- adattare  
                continue  
            # Si puliscono i campi uno per uno (P6), non la riga intera.  
            campi = []  
            for campo in riga.split(SEPARATORE):  
                campi.append(campo.strip())  
            righe.append((numero, campi))  
    return righe
```

10.2 P26 in azione — Un lotto con un commento, una vuota e due storte

```
riga 2: ['LS-0701', 'NORD', '12.5']
riga 3: ['LS-0702', 'sud', '7.2']
riga 5: ['LS-0703', 'ISOLE', '3.0']
riga 6: ['LS-0704', 'CENTRO']
riga 7: ['LS-0705', 'NORD', 'venti']
```

Mancano la 1 (commento) e la 4 (vuota): buchi voluti, dicono dove andare. TRABOCCHETTO: con un contatore += 1 dopo il continue, LS-0704 diventa la riga 4, e chi apre l'editor corregge la riga sbagliata. Con un'intestazione: enumerate(f, start=2).

10.3 P27 — Il dizionario di riferimento da file, con i default

```
import json

def carica_listino(percorso):
    """Da righe CHIAVE;valore a dizionario {chiave: float}."""
    listino = {}
    with open(percorso, encoding="utf-8") as f:
        for riga in f:
            chiave, valore = riga.rstrip("\n").split(";")
            # Chiave normalizzata e valore CONVERTITO: dal file arriva testo.
            listino[chiave.strip().upper()] = float(valore) # <-- adattare
    return listino

def carica_configurazione(percorso, default):
    """Legge un JSON e completa con i default le chiavi che mancano."""
    with open(percorso, encoding="utf-8") as f:
        letta = json.load(f)
    configurazione = {}
    # Si scorrono i DEFAULT: vale il file se dichiara la chiave, se no il default.
    for chiave, valore in default.items():
        configurazione[chiave] = letta.get(chiave, valore)
    return configurazione
```

11. SCRIVERE SUI FILE

P28-P31 · Ogni prodotto in output/, e due esecuzioni di fila lo lasciano identico

11.1 P28 — Il report composto una volta, a video e su file

```
def componi_report(dati): # <-- adattare: i dati che servono al report
    """Restituisce le righe del report come lista di stringhe. Non stampa."""
    righe = ["TITOLO DEL REPORT"] # <-- adattare
    for chiave, valore in dati.items():
        righe.append(f"{chiave:<14}{valore:>10.2f}") # <-- adattare: P2, P3
    return righe

def stampa_righe(righe, file=None):
    """Manda le righe a video (file=None, il default di print) o sul file."""
    for riga in righe:
        print(riga, file=file)
```

Quando serve: lo stesso report a schermo e in un file. Scritto due volte, con print() e con write(), al primo ritocco le versioni divergono: si compone una volta, come lista di stringhe, e si manda dove serve.

11.1 P28 in azione — Le pratiche di Villanova, a video e su file

```
PRATICHE EVASE PER UFFICIO
```

```
ANAGRAFE          42
```

```
TRIBUTI           18
```

```
EDILIZIA          9
```

```
Totale            69
```

```
Righe a video 5, righe nel file 5
```

```
File identico al video: True
```

Le ultime due righe sono il collaudo del proprio output: si rilegge il file e si confronta. TRABOCCHETTO: con `f.write(riga)` al posto di `print(riga, file=file)` le cinque righe finiscono incollate su una sola, e il confronto dice False.

11.2 P29 — Il registro e il file degli scarti: modalità a e ripartenza

```
def apri_registro(percorso):  
    """Ripartenza pulita: svuota il registro all'avvio del programma."""  
    # write_text apre in "w": il file si crea vuoto, o si svuota se c'era.  
    percorso.write_text("", encoding="utf-8")  
  
def registra(percorso, numero, livello, messaggio):  
    """Accoda una riga NNN | LIVELLO | messaggio. Livelli: INFO, SCARTO, ERRORE."""  
    with open(percorso, "a", encoding="utf-8") as f:  
        f.write(f"{numero:03d} | {livello:<6} | {messaggio}\n")
```

Quando serve: ogni programma che scarta righe o deve lasciare traccia di sé. "w" una volta all'avvio, "a" a ogni evento: il file racconta QUESTA esecuzione. Il numero è quello della riga d'origine, 0 per gli eventi del programma.

11.2 P29 in azione — Due esecuzioni di fila, lo stesso registro

```
002 | SCARTO | peso non numerico: 'venti'  
004 | SCARTO | peso non numerico: ''  
000 | INFO    | fine: 4 righe lette  
Due esecuzioni, stesso registro: True
```

Niente orario nel registro: cambia a ogni lancio, e il collaudo non si può fare. TRABOCCHETTO: togliete `apri_registro()` e alla seconda esecuzione il registro ha sei righe, con 002 e 004 due volte ciascuna. È E8.

11.3 P30 — JSON andata e ritorno: carica, modifica, salva in output/

```
import json
```

```
def carica_json(percorso):
```

```
    """Restituisce il contenuto del file JSON: una lista o un dizionario."""
```

```
    with open(percorso, encoding="utf-8") as f:
```

```
        return json.load(f)
```

```
def salva_json(percorso, dati):
```

```
    """Scrivi i dati in JSON leggibile: rientri di 2, lettere accentate vere."""
```

```
    # Il percorso di arrivo sta in output/, MAI sopra il file di dati/.
```

```
    with open(percorso, "w", encoding="utf-8") as f:
```

```
        json.dump(dati, f, indent=2, ensure_ascii=False)
```

11.4 P31 — Il riepilogo di una cartella: glob e sorted()

```
def righe_per_file(cartella, modello): # <-- adattare: cartella e modello
    """Restituisce [(nome, righe), ...] dei file del modello, ordinata per nome."""
    risultato = []
    # sorted(): glob da' i file nell'ordine del disco, diverso fra macchine.
    for percorso in sorted(cartella.glob(modello)):
        with open(percorso, encoding="utf-8") as f:
            risultato.append((percorso.name, len(f.readlines())))
    return risultato
```

Quando serve: tanti file dello stesso tipo, uno per mese, negozio o sede, e una domanda sull'insieme. Il modello del nome sceglie i file; quale file NON è arrivato glob non può dirlo: serve una lista degli attesi, P21, e poi P33.

11.4 P31 in azione — Tre file, uno vuoto, una nota da non contare

```
a.txt      3
b.txt      0  [!] vuoto
c.txt      2
File 3, righe in tutto 5
```

Il file vuoto non è un errore, ma va detto: è spesso un invio mancato. TRABOCCHETTO: senza sorted() l'ordine è quello del disco, diverso su due macchine; con "" al posto di "*.txt" entra anche note.md: 4 file, 6 righe.*

12. GLI ERRORI PREVISTI

P32-P33 · Non sono difetti del programma: il programma arriva in fondo

12.1 P32 — La riga che non si converte: try dentro il ciclo

```
buoni = []
scarti_per_tipo = {}
for campi in righe: # <-- adattare: le righe gia' spezzate
    # Nel try SOLO le righe che possono sollevare: le conversioni.
    try:
        valore = float(campi[1]) # <-- adattare
    except (ValueError, IndexError) as e:
        # Il tipo come testo diventa la chiave di un contatore (P17).
        tipo = type(e).__name__
        scarti_per_tipo[tipo] = scarti_per_tipo.get(tipo, 0) + 1
        continue
    # Il calcolo sta FUORI: un suo errore e' un difetto, e deve vedersi.
    buoni.append((campi[0], valore)) # <-- adattare
assert len(righe) == len(buoni) + sum(scarti_per_tipo.values()), "righe perse"
```

12.1 P32 in azione — Sei ordini NovaStore, due righe storte

```
[!] riga 2: invalid literal for int() with base 10: 'due'  
[!] riga 4: could not convert string to float: ''  
Lette 6, buone 4, scartate 2: {'ValueError': 2}  
Totale ordini buoni: 385.40
```

TRABOCCHETTO: con il try fuori dal for la riga 2 ferma la lettura, «Lette 2, buone 1, scartate 1», e l'assert NON scatta, perché i conti delle righe lette tornano. L'assert controlla ciò che il programma ha visto, non ciò che non ha visto.

12.2 P33 — Il file che manca o non si legge: un except per causa

```
def leggi_testo(percorso):  
    """Restituisce il testo del file, oppure "" se non c'è. Mai un traceback."""  
    try:  
        return percorso.read_text(encoding="utf-8")  
    # Un except per causa, dal particolare al generale (Giorno 09 §7.2).  
    except FileNotFoundError:  
        print(f"[ERRORE] manca {percorso.name}: chiedetelo a chi lo invia")  
    except UnicodeDecodeError:  
        # Il ripiego: seconda lettura nella codifica dei gestionali vecchi.  
        print(f"[!] {percorso.name} non è in utf-8: riletto in latin-1")  
        return percorso.read_text(encoding="latin-1") # <-- adattare  
    return ""
```

Quando serve: il file arriva da un altro ufficio e può mancare, essere in un'altra codifica, essere malformato. Ogni causa ha il suo except, il suo messaggio e, se possibile, un ripiego. Il ripiego si dichiara: silenzioso, nessuno corregge la causa.

12.2 P33 in azione — Un file mancante, uno in latin-1, un JSON rotto

```
[ERRORE] manca tariffe.txt: chiedetelo a chi lo invia
    tariffe.txt: 0 caratteri
[!] anagrafe_vecchia.txt non è in utf-8: riletto in latin-1
    anagrafe_vecchia.txt: 11 caratteri
[ERRORE] config.json alla riga 3: Expecting ',' delimiter
Arrivato in fondo. Configurazione: {}
```

Il parser dice riga 3, e la virgola manca alla fine della riga 2: guardate la riga indicata e quella sopra. TRABOCCHETTO: un except OSError scritto sopra FileNotFoundError prende anche il file mancante, perché ne è la madre.

13. LO SCRIPT COMPLETO

P34 · Non un pezzo nuovo: il montaggio dei pezzi che esistono

13.1 P34 — Carica, valida, elabora, riporta: lo scheletro

```
# ===== PARTE GENERALE: si riusa cosi' com'e' =====

# Le funzioni dei pattern con le firme di sempre: non conoscono il caso, e si
# copiano identiche. carica_righe (P26), conta_per_chiave (P17),
# somma_per_chiave (P18), stampa_righe (P28).

# ===== PARTE SPECIFICA: si cambia per un altro caso =====

CATEGORIE = ["PRIMA", "SECONDA"] # <-- adattare: costanti, validatore, report

def main():

    # 1. CARICA: dal file alle righe numerate (P25, P26).

    # 2. VALIDA: ogni riga fra i buoni o fra gli scarti (P20, P24, P32).

    # 3. ELABORA: contare, sommare, massimo, classifica (P15-P18, P22).

    # 4. RIPORTA: report a video e su file, scarti, invariante (P28, P29).

    assert len(righe) == len(buoni) + len(scarti), "righe perse nel conto"
```

13.1 P34 in azione — I resi NovaStore: carica e valida (1/2)

```
# ===== PARTE SPECIFICA (estratto): il regista =====

def main():

    prepara_file_di_prova()

    OUTPUT.mkdir(parents=True, exist_ok=True)

    # 1. CARICA

    righe = carica_righe(PERCORSO_RESI)

    # 2. VALIDA: ogni riga fra i resi o fra gli scarti, con il suo motivo.

    resi = []

    scarti = []

    for numero, campi in righe:

        reso, motivo_scarto = valida_reso(campi)

        if motivo_scarto:

            print(f"[!] riga {numero}: {motivo_scarto}")

            scarti.append(f"{numero};{motivo_scarto};{SEPARATORE.join(campi)}")

        else:

            resi.append(reso)
```

13.1 P34 in azione — I resi NovaStore: elabora e riporta (2/2)

```
# 3. ELABORA: le chiavi per P17, le coppie per P18, il massimo con P16.

motivi = []

coppie = []

piu_alto = resi[0]

for reso in resi:

    motivi.append(reso[1])

    coppie.append((reso[1], reso[2]))

    if reso[2] > piu_alto[2]:

        piu_alto = reso

conteggi = conta_per_chiave(motivi)

somme = somma_per_chiave(coppie)

# 4. RIPORTA: lo stesso report a video e su file, poi gli scarti.

report = componi_report(conteggi, somme, piu_alto)

stampa_righe(report)

with open(OUTPUT / "report_resi.txt", "w", encoding="utf-8") as f:

    stampa_righe(report, file=f)

with open(OUTPUT / "scarti_resi.txt", "w", encoding="utf-8") as f:

    print("riga;motivo;contenuto", file=f)

    stampa_righe(scarti, file=f)

for percorso in sorted(OUTPUT.glob("*.txt")):

    quante = len(percorso.read_text(encoding="utf-8").splitlines())

    print(f"File scritto: {percorso.name} ({quante} righe)")

assert len(righe) == len(resi) + len(scarti), "righe perse nel conto"

print(f"[OK] righe utili {len(righe)} = resi {len(resi)} + scarti {len(scarti)}")
```

13.1 P34 in azione — L'output dei resi

```
[!] riga 6: importo non numerico: 'dodici'
[!] riga 8: motivo sconosciuto: SCONTENTO
RESI PER MOTIVO
DIFETTOSO      2      268.90    69.5%
RIPENSAMENTO   2       81.00    20.9%
ERRATO         2       36.90     9.5%
Totale         6      386.80
Reso più alto: NS-R03 (DIFETTOSO) 219.00
File scritto: report_resi.txt (6 righe)
File scritto: scarti_resi.txt (3 righe)
[OK] righe utili 8 = resi 6 + scarti 2
```

TRABOCCHETTO: se carica_righe() facesse anche .upper() sul secondo campo, qui funzionerebbe lo stesso; ricopiata nell'anagrafe di Villanova scriverebbe «DE LUCA» dove il file dice «De Luca». La logica del caso NON entra nella parte generale.

13.1 P34 — La tabella di adattamento: otto righe, tutte specifiche

Riga della parte specifica	Resi NovaStore	Reclami Aurora	Interventi Villanova
MOTIVI	DIFETTOSO, RIPENSAMENTO, ERRATO	ATTESA, PERSONALE, PULIZIA	STRADE, ILLUMINAZIONE, VERDE
IMPORTO_MASSIMO	1000.00 euro	50.00 euro di buono	8.0 ore
PERCORSO_RESI	resi.txt	reclami.txt	interventi.txt
OUTPUT	output/resi	output/reclami	output/interventi
messaggi in valida_reso()	importo non numerico	buono non numerico	ore non numeriche
titolo in componi_report()	RESI PER MOTIVO	RECLAMI PER CATEGORIA	ORE PER TIPOLOGIA
riga del massimo in componi_report()	Reso più alto	Buono più alto	Intervento più lungo
nomi dei file in main()	report_resi.txt	report_reclami.txt	report_interventi.txt

La parte generale non si apre nemmeno. I nomi MOTIVI, IMPORTO_MASSIMO, valida_reso possono restare: rinominarli è una seconda passata di pulizia. L'adattamento sono le otto righe, e chi adatta ne scrive l'elenco in testa.

14. ERRORI TIPICI DELLA GIORNATA

Errori di montaggio: il programma gira, il report esce, ed è sbagliato

E1-E5 Errori di montaggio: sintomo e cura (1/2)

Errore	Sintomo, senza traceback	Cura
E1 Lo schema copiato senza toccare le righe # <-- adattare (P34)	RESI PER MOTIVO {}: tre ticket letti, nessuno contato	Cercare tutte le righe marcate e scriverne l'elenco in testa
E2 Il contatore azzerato dentro il ciclo (P15, P17)	{'REFERTO': 1}: vale 1 invece di 3, PAGAMENTO è sparito	L'accumulatore nasce fuori dal ciclo, numero, lista o dizionario
E3 La soglia con >= al posto di > (P10)	Ritardi 4 su 5 invece di 2: i casi sul bordo cambiano fascia	Il bordo si decide leggendo la regola, e si collauda con un assert
E4 in che trova una parola dentro un'altra (P11)	«Pioggia a diretto»: 3 punti di urgenza senza nessun guasto	Dividere il testo in parole e cercarle con .get(parola, 0)
E5 Il doppione contato due volte (P21)	Compatibilità 133.3%: il numeratore conta ciò che il denominatore non conta	Scorrere le RICHIESTE fra le voci normalizzate: 2 su 3, 66.7%

Gli errori del singolo pattern stanno nella sua scheda; questi nascono mettendo insieme i pezzi. Una quota oltre il 100% è sempre un sintomo. Il capitolo è il pronto soccorso dei laboratori.

E6-E9 Errori di montaggio: sintomo e cura (2/2)

Errore	Sintomo, senza traceback	Cura
E6 La parità nella classifica: reverse=True rovescia anche i nomi (P22)	Il terzo posto a 75 va a Greco solo perché la G viene dopo la B	Punteggio con il segno meno e ordine crescente, o l'ordine dichiarato
E7 Il try troppo largo (P32)	Buone 0, scartate 3: un difetto del programma registrato come scarto	Nel try le conversioni e basta; l'append nell'else o dopo
E8 Il file in output/ che cresce: "a" senza ripartenza pulita (P29)	Lancio 1: 2 righe, lancio 2: 4, lancio 3: 6, con gli stessi dati	apri_registro() di P29: "w" all'avvio, "a" per ogni evento
E9 Il percorso relativo che funziona solo da PyCharm (P25)	Da progetto trovato: True, da altrove trovato: False	Path(__file__).resolve().parent, che non dipende da dove lanciate

E7 è il più insidioso: append riceve due argomenti invece di una tupla, il TypeError è un difetto del programma, e il try largo lo registra come riga scartata. Il file era sano per due terzi.

E6 La parità nella classifica: chi entra fra i primi tre?

```
PUNTEGGI = {"Amato Chiara": 82, "Bruno Dario": 75, "Caruso Elisa": 90,
            "Greco Lorenzo": 75}

coppie = []

for nome, punteggio in PUNTEGGI.items():
    coppie.append((punteggio, nome))

coppie.sort(reverse=True)

print("reverse=True:", coppie[:3])

# Il rimedio: il punteggio con il segno meno e l'ordine crescente. Il piu'
# alto diventa il piu' piccolo, e a parita' i nomi restano in ordine A-Z.

coppie = []

for nome, punteggio in PUNTEGGI.items():
    coppie.append((-punteggio, nome))

coppie.sort()

print("segno meno: ", coppie[:3])

# reverse=True: [(90, 'Caruso Elisa'), (82, 'Amato Chiara'), (75, 'Greco Lorenzo')]
# segno meno:  [(-90, 'Caruso Elisa'), (-82, 'Amato Chiara'), (-75, 'Bruno Dario')]
```

Riepilogo del giorno (1/2)

L'uscita — P1 main() regista e guardia · P2 .2f per i soldi, .1% sulla frazione

P3 larghezze in costanti, separatore calcolato, la fetta per il testo che sfonda

L'ingresso — P4 la stessa normalizzazione su tutti e due i lati del confronto

P5 se c'è la virgola: via i punti, poi la virgola diventa punto

P6 i campi si contano PRIMA dell'unpacking e si puliscono uno per uno

Validare — P7 presenza, forma, intervallo, ammessi: in quest'ordine

P8 la funzione solleva, chi chiama decide · P9 try, continue e un tetto

Decidere — P10 soglie crescenti e $\leq$, collaudo sui bordi

P11 in trova sequenze, non parole · P12 .get(chiave, ripiego) su chiave normalizzata

Ripetere — P13 lettura prima del ciclo e in coda al corpo · P14 una funzione per voce

Accumulare — P15 il caso zero lo controlla chi chiama · P16 si parte dal primo

P17 conteggi.get(k, 0) + 1 · P18 + importo al posto di + 1, la quota dopo il totale

Riepilogo del giorno (2/2)

Cercare — P19 return al primo che combacia, None dopo il for

P20 i buoni in una lista, gli scarti con il motivo · P21 la lista dei già visti

P22 coppie (valore, nome), sort(reverse=True), i primi N: attenzione alla parità

Funzioni — P23 calcola_ restituisce, stampa_ stampa · P24 (record, "") o (None, motivo)

Leggere — P25 Path(__file__) · P26 il numero di riga dell'editor · P27 i default

Scrivere — P28 righe composte una volta · P29 "w" all'avvio, "a" durante

P30 si salva in output/, mai sopra i dati · P31 glob sempre dentro sorted()

Errori previsti — P32 try dentro il for, assert sui conti · P33 un except per causa

Script completo — P34 carica, valida, elabora, riporta; generale e specifico separati

Gli errori di montaggio quasi mai danno un traceback: il report esce, ed è sbagliato.

I più usati — P26, P18, P22 — reggono ogni programma che legge un file

e ne tira fuori un numero.

Grazie.

shaping the skills of tomorrow

